NYCU Introduction to Machine Learning, Homework 3

[112705007], [解孟修]

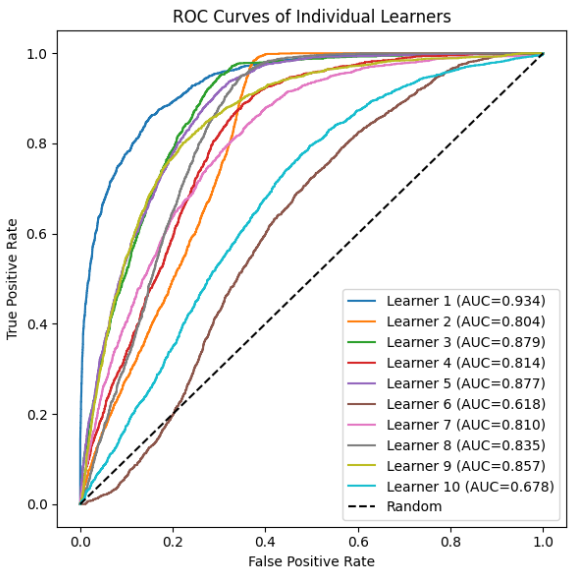
Part. 1, Coding (60%):

(20%) Adaboost

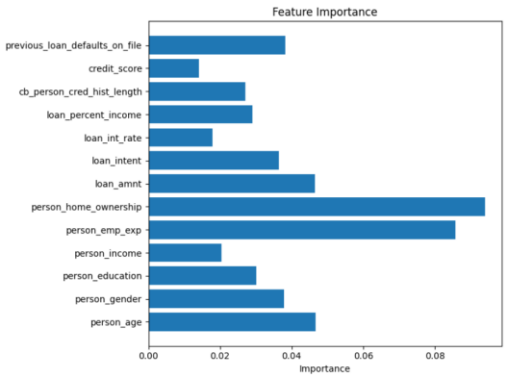
1. (5%) Show your accuracy of the testing data (n_estimators = 10)

```
2025-11-18 13:31:36.002 | INFO     | __main__:main:49 - Num features: 13
2025-11-18 13:31:36.049 | INFO     | __main__:main:63 - Training AdaBoost ...
2025-11-18 13:31:46.245 | INFO     | __main__:main:74 - AdaBoost - Accuracy: 0.8740
```

2. (5%) Plot the AUC curves of each weak classifier.



3. (10%)
   a. Plot the feature importance of the AdaBoost method.



   b. Paste the snapshot of your implementation of the feature importance estimation.

```python
def compute_feature_importance(self) -> t.Sequence[float]:
    if not self.learners:
        return []

    first_weight = next(self.learners[0].parameters()).detach().cpu().numpy()
    n_features = first_weight.shape[-1]

    importance = np.zeros(n_features, dtype=np.float64)

    for alpha_t, learner in zip(self.alphas, self.learners):
        w = learner.linear.weight.detach().cpu().numpy().reshape(-1)
        importance += abs(alpha_t) * np.abs(w)

    if importance.sum() > 0:
        importance = importance / importance.sum()
    return importance
```

c. Explain how you compute the feature importance for the Adaboost method shortly (within 100 words as possible)
Each weak learner contributes a weight α_t based on its error rate(Formula 1). We set a vector which value is $\alpha_t \cdot |\mathbf{w}_t|$, and $|\mathbf{w}_t|$ is the absolute value of the linear model's weights. The final feature importance is computed by summing $\alpha_t \cdot |\mathbf{w}_t|$ over all learners and then normalizing the result.

$$\alpha_t = \frac{1}{2} \ln \frac{1 - \epsilon_t}{\epsilon_t}$$

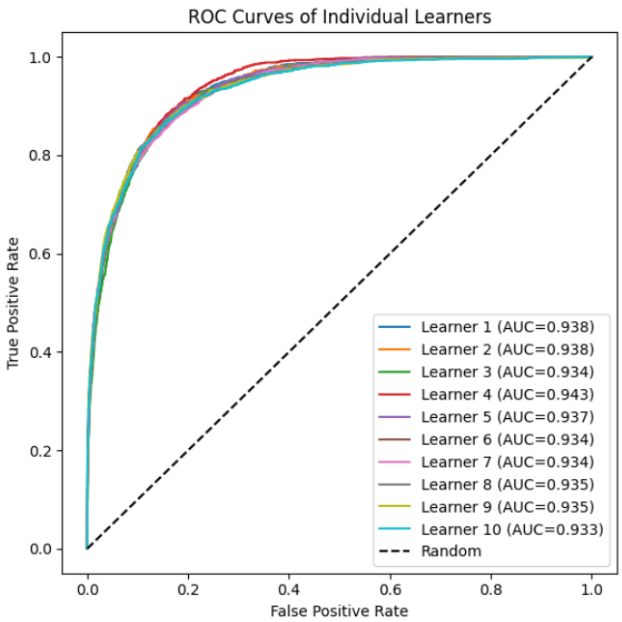
formula 1

(20%) Bagging

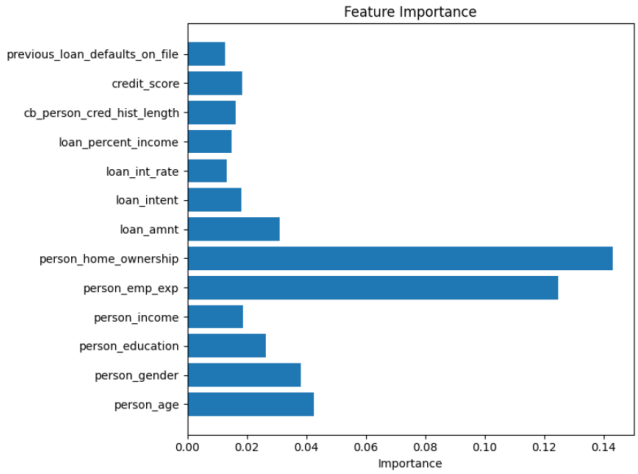
4. (5%) Show the accuracy of the test data using 10 estimators. (n_estimators=10)

```
2025-11-18 14:46:53.083 | INFO     | __main__:main:99 - Bagging - Accuracy: 0.8846
```

5. (5%) Plot the AUC curves of each weak classifier.



6. (10%)
   a. Plot the feature importance of the Bagging method.



b. Paste the snapshot of your implementation of the feature importance estimation.

```python
def compute_feature_importance(self) -> t.Sequence[float]:
    if not self.learners:
        return []

    first_weight = next(self.learners[0].parameters()).detach().cpu().numpy()
    n_features = first_weight.shape[-1]

    importance = np.zeros(n_features, dtype=np.float64)

    for learner in self.learners:
        w = learner.linear.weight.detach().cpu().numpy().reshape(-1)
        importance += np.abs(w)

    importance /= len(self.learners)
    if importance.sum() > 0:
        importance = importance / importance.sum()
    return importance
```

c. Explain how you compute the feature importance for the Bagging method shortly (within 100 words as possible)

For each learner, we get its linear weight vector and take the absolute value. We accumulate these value across all learners and then divide by the number of learners to obtain the averaged importance.

Finally, we normalize the vector so that the importances sum to 1.

(15%) Decision Tree

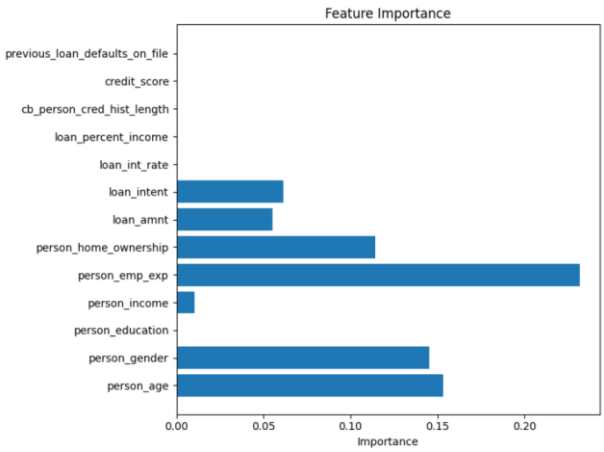
7. (5%) Compute the Gini index and the entropy of the array [0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1].

   Gini index: 0.496, entropy: 0.992

8. (5%) Show your accuracy of the testing data with a max-depth = 7

```
2025-11-18 14:48:58.125 | INFO     | __main__:main:118 - DecisionTree - Accuracy: 0.9146
```

9. (5%) Plot the feature importance of the decision tree.



(5%) Code Linting

10. Show the snapshot of the flake8 linting result (paste the execution command even when there is no error).

```
PS C:\Users\matth\OneDrive\桌面\機器學習\hw3> python -m flake8 main.py
PS C:\Users\matth\OneDrive\桌面\機器學習\hw3>
```

Part. 2, Questions (40%):

1. (15%)

In the AdaBoost algorithm, each selected weak classifier, h_t is assigned a weight:

$$\alpha_t = \frac{1}{2} ln \frac{1-\epsilon_t}{\epsilon_t}$$

(a) What is the definition of ϵ_t in this formula?

(b) If a weak classifier h_t performs only as well as "random guessing" (i.e., $\epsilon_t = 0.5$), what will α_t be?

(c) Based on your answer in (b), briefly explain how this α_t weight formula ensures the robustness of AdaBoost.

(a) ϵ_t is the weighted error rate of the classifier, which defined as below

$$\varepsilon_t = \sum_{i=1}^{N} w_i^{(t)} \cdot \mathbf{1}(h_t(x_i) \neq y_i)$$

Where:
W_i = sample weight at iteration t
H_t = prediction of classifier t
Y_i = true label
The indicator = 1 if prediction is wrong, 0 otherwise
Therefore, ϵ_t measures how much weighted mass of samples the classifier misclassified
(b) α_t will be 0

(c) When $\epsilon_t = 0.5$ ($\alpha_t = 0$), this classifier is a random classifier, and the weak classifier contributes no influence to the final ensemble.(If $\epsilon_t > 0.5$(negative α_t) the vote of classifier will be reversed to the final ensemble)

2. (15%) Random Forest is often considered an improvement over Bagging (Bootstrap Aggregating) when used with Decision Trees as the base learners.

   (a) Briefly explain the potential problem that can exist among the T classifiers (C_1, ..., C_ T) produced by Bagging when using decision trees.

   (b) To mitigate the problem from (a), Random Forest introduces a second source of randomness in addition to Bagging. Specifically describe what this second source of randomness is and how it works.

   (a) If the dataset contains strong predictive features, many trees may repeatedly select the same features for splitting, resulting in similar tree structures. Such correlation limits the variance reduction effect of Bagging and weaken the performance of the ensemble.
   (b) While bagging builds each decision tree using all features, Random Forest uses feature subsampling, It restricts each split to a random subset of features. Within this subset, it chooses the best possible split, ensuring that each tree remains strong while increasing diversity across trees.

3. (10%) Is it always possible to find the smallest decision tree that codes a dataset with no error in polynomial time? If not, what algorithm do we usually use to search a decision tree in a reasonable time? Also, list the criterion that we stop splitting the decision tree and leaf nodes are created (List two).

   No, it is not possible to guarantee finding the smallest decision tree in polynomial time. The problem of constructing the minimum-sized decision tree is known to be NP-complete, so an exact solution is computationally infeasible for realistic datasets. We use greedy top-down algorithms such as ID3, C4.5, or CART, which choose the best split at each node based on impurity measures (ex: information gain or Gini index). These methods do not guarantee the optimal tree but can construct a good tree efficiently.
   Typical stopping criteria include:
   All samples at the node belong to the same class (no impurity).
   The maximum depth is reached, or the number of samples at the node is too small to continue splitting.